Limite inferior da ordenação por comparaç

Universidade de Fortaleza

Todos os algoritmos vistos até aqui obtêm informação **comparando** pares de elementos. Quantas comparações são necessárias no pior caso?

- Um vetor de n elementos distintos admite $n!$ permutações;
- Cada comparação divide o espaço de possibilidades em duas partes (árvore de decisão binária);
- Com k comparações distinguimos no máximo 2^k permutações.

Logo, é preciso ter $2^k \geq n!$. Como $n! \geq (n/2)^{n/2}$:

$$k \geq \log_2 n! \geq \frac{n}{2} \log_2 \frac{n}{2} = \Omega(n \log n)$$

Ideia-chave

Nenhum algoritmo baseado em comparações pode ser melhor que $\Omega(n \log n)$. Mergesort e Heapsort são, portanto, **assintoticamente ótimos**.

O que é o Counting Sort?

- É um algoritmo de ordenação **por contagem**: em vez de comparar elementos entre si, ele conta **quantas vezes** cada valor aparece no vetor.
- Funciona quando os elementos são inteiros dentro de um intervalo **conhecido** $[0, k]$.
- Usa essas contagens para calcular diretamente a **posição final** de cada elemento no vetor ordenado.

Diferença fundamental em relação aos algoritmos anteriores

O Counting Sort **não divide** o problema pela metade (como MergeSort/QuickSort) nem **decresce** o problema em 1 (como Seleção/Bubble/Insertion). Ele simplesmente **percorre os dados um número fixo de vezes**. Por isso, **não existe uma formulação recursiva natural** para esse algoritmo — ele é, por essência, iterativo.

Ideia geral do algoritmo

O Counting Sort é feito em **quatro fases**:

1. **Contar** quantas vezes cada valor $0, 1, \ldots, k$ aparece em v;
2. Transformar essas contagens em **somas acumuladas**, obtendo quantos elementos são $\leq$ cada valor;
3. Usar as somas acumuladas para colocar cada elemento de v diretamente em sua **posição final** em um vetor de saída;
4. Copiar o vetor de saída de volta para v (se necessário).

Cada fase é implementada com um **laço simples**, sem chamadas recursivas.

Pseudocódigo do Counting Sort (Iterativa)

Universidade de Fortaleza

Entrada: vetor v de tamanho n, com valores em $[0, k]$
Saída : v ordenado

```
1  função countingSort(v, n, k):
2      C ← vetor de tamanho k + 1, todo zerado;
3      for i ← 0 to n − 1 do
4          C[v[i]] ← C[v[i]] + 1 ;                  // conta ocorrências
5      for j ← 1 to k do
6          C[j] ← C[j] + C[j − 1] ;                 // soma acumulada
7      B ← vetor de tamanho n ;                     // saída
8      for i ← n − 1 to 0 do
9          B[ C[v[i]] − 1 ] ← v[i];
10         C[v[i]] ← C[v[i]] − 1;
11     copia B para v;
```

Exemplo passo a passo

Vamos simular o Counting Sort no seguinte vetor:

$$L = [\,4,\ 2,\ 2,\ 8,\ 3,\ 3,\ 1\,]$$

O algoritmo segue 5 passos:

1. encontrar o maior elemento da lista;
2. criar um vetor H de contagem, do tamanho $\max + 1$, zerado;
3. contar as ocorrências de cada valor em H;
4. transformar H em soma acumulada, obtendo C;
5. usar C para colocar cada elemento de L em sua posição final em S.

Passo 1. Encontrar o maior elemento da lista.

$$L = [4, 2, 2, 8, 3, 3, 1] \qquad \max = 8$$

Passo 2. Criar um vetor H de tamanho $\max + 1 = 9$, todo zerado.

$$H = [\,0, 0, 0, 0, 0, 0, 0, 0, 0\,]$$

índices: 0 1 2 3 4 5 6 7 8

Passo 3: contar as ocorrências

Para cada elemento de L, incrementamos a posição correspondente em H:

$$L = [4, 2, 2, 8, 3, 3, 1]$$

$$H = [\,0,\ 1\ ,\ 2\ ,\ 2\ ,\ 1\ ,\ 0,\ 0,\ 0,\ 1\ ]$$

índices: 0 1 2 3 4 5 6 7 8

O valor 2 aparece 2 vezes $\Rightarrow H[2] = 2$; o valor 8 aparece 1 vez $\Rightarrow H[8] = 1$; e assim por diante.

Passo 4: soma cumulativa

Cada posição de C soma todos os valores de H até ali — ou seja, $C[j]$ diz quantos elementos de L são $\leq j$:

$$H = [0, 1, 2, 2, 1, 0, 0, 0, 1]$$

$$C = [\,0,\ 1,\ 3,\ 5,\ 6,\ 6,\ 6,\ 6,\ 7\,]$$

índices:	0	1	2	3	4	5	6	7	8

Por exemplo, $C[4] = 6$: existem 6 elementos em L menores ou iguais a 4.

Passo 5: construir a saída

Para cada elemento de L (da direita para a esquerda, garantindo estabilidade), colocamos $L[i]$ na posição $C[L[i]]-1$ de S, e decrementamos $C[L[i]]$.

i	$L[i]$	$C[L[i]]$	posição $= C[L[i]]-1$	S após o passo
6	1	1	0	$[1,_,_,_,_,_,_]$
5	3	5	4	$[1,_,_,_,3,_,_]$
4	3	4	3	$[1,_,_,3,3,_,_]$
3	8	7	6	$[1,_,_,3,3,_,8]$
2	2	3	2	$[1,_,2,3,3,_,8]$
1	2	2	1	$[1,2,2,3,3,_,8]$
0	4	6	5	$[1,2,2,3,3,4,8]$

$$L = [4, 2, 2, 8, 3, 3, 1]$$

Vetor ordenado:

$$S = [\,1,\ 2,\ 2,\ 3,\ 3,\ 4,\ 8\,]$$

Note que o Counting Sort nunca comparou dois elementos entre si — a ordenação surgiu inteiramente das contagens em H e C.

Como não há recorrência, analisamos diretamente o custo de **cada laço**:

Fase	**Custo**
Inicializar o vetor C (tamanho $k+1$)	$\Theta(k)$
Contar ocorrências (percorre v)	$\Theta(n)$
Calcular somas acumuladas (percorre C)	$\Theta(k)$
Construir a saída B (percorre v)	$\Theta(n)$
Copiar B de volta para v	$\Theta(n)$

Cada fase é um laço **independente**; o tempo total é a **soma** dos custos de todas elas.

Somando o custo de todas as fases:

$$f(n,k) = \underbrace{\Theta(k)}_{\text{inicializar } C} + \underbrace{\Theta(n)}_{\text{contar}} + \underbrace{\Theta(k)}_{\text{acumular}} + \underbrace{\Theta(n)}_{\text{construir } B} + \underbrace{\Theta(n)}_{\text{copiar}}$$

Somando o custo de todas as fases:

$$f(n,k) = \underbrace{\Theta(k)}_{\text{inicializar } C} + \underbrace{\Theta(n)}_{\text{contar}} + \underbrace{\Theta(k)}_{\text{acumular}} + \underbrace{\Theta(n)}_{\text{construir } B} + \underbrace{\Theta(n)}_{\text{copiar}}$$

Tempo total

$$T(n,k) = \Theta(n+k)$$

O tempo depende de **dois** parâmetros: o tamanho do vetor (n) e a amplitude dos valores (k).

$$T(n) = \Theta(n + k)$$

- Se $k = O(n)$ (valores dentro de uma faixa proporcional ao tamanho do vetor), então $T(n) = \Theta(n)$ — **mais rápido** que qualquer algoritmo baseado em comparações;
- Isso não contradiz o limite inferior $\Omega(n \log n)$ dos algoritmos de ordenação por comparação, pois o Counting Sort **nunca compara** dois elementos entre si;
- Se k for muito maior que n (ex.: $k = n^2$), o algoritmo se torna **ineficiente**, pois gasta $\Theta(k)$ apenas percorrendo o vetor de contagem.

Por que não há relação de recorrência?

- **MergeSort, QuickSort, Heap Sort**: *dividem* o problema em partes menores → recorrência do tipo $T(n) = a \cdot T(n/b) + f(n)$;
- **Seleção, Bubble, Insertion Sort**: *decrescem* o problema em 1 a cada passo → recorrência do tipo $T(n) = T(n-1) + f(n)$;
- **Counting Sort**: *não divide nem decresce* o problema — ele percorre a entrada inteira um número **fixo** de vezes.

Conclusão

Sem chamadas recursivas, não existe uma relação de recorrência a construir. A complexidade é obtida somando diretamente o custo dos laços — a mesma técnica usada para analisar as versões **iterativas** dos algoritmos anteriores.

	Baseado em comparação (MergeSort, QuickSort, ...)	**Counting Sort**
Tempo (pior caso)	$\Omega(n \log n)$	$\Theta(n+k)$
Compara elementos?	Sim	Não
Requer conhecer o intervalo?	Não	Sim ($[0,k]$)
Espaço extra	varia	$\Theta(n+k)$

O Counting Sort **troca** a generalidade (funcionar com qualquer tipo comparável) por **velocidade**, aproveitando uma característica específica dos dados: valores inteiros em um intervalo conhecido.

- O Counting Sort ordena **contando ocorrências** de cada valor, sem comparar elementos.
- É executado em **quatro fases**, cada uma implementada com um laço simples — não há recursão.
- Complexidade obtida por **soma direta** dos custos dos laços (e não por árvore de recursão):

$$T(n) = \Theta(n + k)$$

- Quando $k = O(n)$, o algoritmo é $\Theta(n)$ — mais rápido que os algoritmos de comparação, que têm limite inferior $\Omega(n \log n)$.
- Só é vantajoso quando o intervalo de valores k não é muito maior que n.

Radix Sort

Generalização do Counting Sort: ordena os números **dígito a dígito**, do menos significativo para o mais significativo, usando um algoritmo estável (em geral o próprio Counting Sort) em cada passagem. Complexidade $\Theta(d \cdot n)$, onde d é o número de dígitos.

$[121,\ 432,\ 564,\ 23,\ 1,\ 45,\ 788] \rightarrow$ unidade $\rightarrow$ dezena $\rightarrow$ centena

Bucket Sort

Distribui os n elementos, supostamente **uniformemente distribuídos**, em n baldes; ordena cada balde e concatena os resultados. Caso médio $\Theta(n)$; pior caso $\Theta(n \log n)$ (todos os elementos no mesmo balde).

Assim como o Counting Sort, ambos **não comparam** elementos entre si – é por isso que escapam do limite $\Omega(n \log n)$.

Algoritmo	Melhor	Médio	Pior
BubbleSort*	$O(n)$	$O(n^2)$	$O(n^2)$
InsertionSort*	$O(n)$	$O(n^2)$	$O(n^2)$
SelectionSort*	$O(n^2)$	$O(n^2)$	$O(n^2)$
ShellSort	$O(n \log n)$	$O(n^{1.25})$ a $O(n \log^2 n)$	$O(n \log^2 n)$ a $O(n^{1.333})$
MergeSort*	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
QuickSort*	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
HeapSort*	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
CountingSort*	$O(n+m)$	$O(n+m)$	$O(n+m)$
RadixSort	$O(nd)$	$O(nd)$	$O(nd)$
BucketSort	$O(n)$	$O(n)$	$O(n \log n)$

* algoritmos vistos em detalhe nesta disciplina (com relação de recorrência, árvore de recursão e/ou exemplo passo a passo).

Algoritmo	Melhor	Médio	Pior
Busca Linear*	$O(1)$	$O(n)$	$O(n)$
Busca Binária*	$O(1)$	$O(\log n)$	$O(\log n)$

* algoritmos vistos em detalhe nesta disciplina.

- A Busca Binária exige um vetor **ordenado**; a Busca Linear funciona em qualquer vetor;
- Essa é a mesma troca vista em vários algoritmos: usar uma **propriedade adicional dos dados** (ordenação, intervalo de valores conhecido) para obter uma complexidade melhor.

- $O(n^2)$ **em todos os casos**: Ordenação por Seleção — sempre percorre o trecho restante inteiro, não importa a entrada;
- $O(n)$ **no melhor caso,** $O(n^2)$ **nos demais**: Bubble Sort e Insertion Sort — se beneficiam de entradas quase ordenadas;
- $O(n \log n)$ **em todos os casos**: MergeSort e Heap Sort — garantia *sempre* boa, independente da entrada;
- $O(n \log n)$ **em média,** $O(n^2)$ **no pior caso**: QuickSort — rápido na prática, mas sensível à escolha do pivô;
- **Sem comparações**: Counting Sort, Radix Sort e Bucket Sort — trocam generalidade por velocidade, aproveitando características específicas dos dados (valores inteiros, intervalo conhecido).

Unidade II

- Simular passo a passo cada algoritmo em um vetor dado;
- Escrever o pseudocódigo (recursivo e iterativo);
- Montar e resolver a relação de recorrência de cada algoritmo;
- Justificar melhor caso, caso médio e pior caso;
- Comparar algoritmos quanto a tempo, espaço e estabilidade;
- Argumentar o limite inferior $\Omega(n \log n)$ e explicar por que o Counting Sort o "contorna".

Laboratório (30%): implementar e comparar empiricamente os algoritmos de ordenação em vetores aleatórios, já ordenados e inversamente ordenados – confrontando os tempos medidos com a previsão teórica da Unidade I.

Dúvidas?

Perguntas? Vamos para a AV2?

